

9. Foliensatz Betriebssysteme

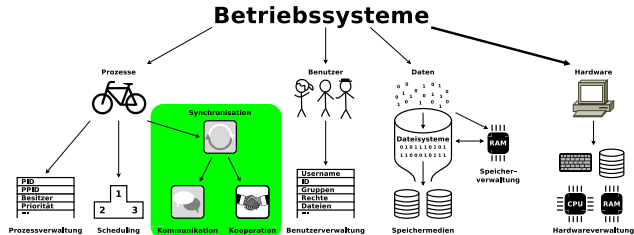
Prof. Dr. Christian Baun

Frankfurt University of Applied Sciences
(1971–2014: Fachhochschule Frankfurt am Main)
Fachbereich Informatik und Ingenieurwissenschaften
christianbaun@fb2.fra-uas.de

Lernziele dieses Foliensatzes

- Am Ende dieses Foliensatzes kennen/verstehen Sie...
 - was **kritische Abschnitte** und **Wettlaufsituationen** sind
 - was **Synchronisation** ist
 - wie **Signalisierung** die Ausführungsreihenfolge der Prozesse beeinflusst
 - wie mit **Blockieren** kritische Abschnitte gesichert werden
 - mögliche Probleme (**Verhungern** und **Deadlocks**) beim Blockieren
 - wie **Deadlock-Erkennung mit Matrizen** funktioniert
 - verschiedene Möglichkeiten der **Kommunikation** zwischen Prozessen:
 - **Gemeinsamer Speicher, Nachrichtenwarteschlangen, Pipes, Sockets**
 - verschiedene Möglichkeiten der **Kooperation** von Prozessen
 - wie **Semaphore** und **Mutexe** kritische Abschnitte sichern können

Übungsblatt 9 wiederholt die für die Lernziele relevanten Inhalte dieses Foliensatzes



Interprozesskommunikation (IPC)

- Prozesse müssen nicht nur Lese- und Schreibzugriffe auf Daten ausführen, sondern auch:
 - sich gegenseitig aufrufen
 - aufeinander warten
 - sich abstimmen
 - kurz gesagt: Sie müssen miteinander **interagieren**
- Bei **Interprozesskommunikation** (IPC) ist zu klären:
 - Wie kann ein Prozess Informationen an andere weiterreichen?
 - Wie können mehrere Prozesse auf gemeinsame Ressourcen zugreifen?

Frage: Wie verhält es sich hier mit Threads?

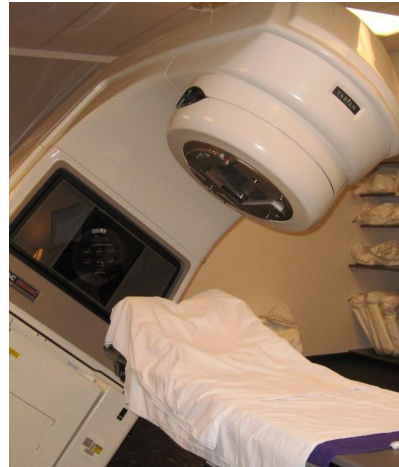
- Bei Threads gelten die gleichen Herausforderungen und Lösungen wie bei Interprozesskommunikation mit Prozessen
- Nur die Kommunikation zwischen den Threads eines Prozesses ist problemlos möglich, weil sie im gleichen Adressraum agieren

Race Condition (*Wettlaufsituation*)

- **Unbeabsichtigte Wettlaufsituation** zweier Prozesse, die den Wert der gleichen Speicherstelle ändern wollen
 - Das Ergebnis eines Prozesses hängt von der Reihenfolge oder dem zeitlichen Ablauf anderer Ereignisse ab
 - Häufiger Grund für schwer auffindbare Programmfehler
- Problem: Das Auftreten und die Symptome hängen von unterschiedlichen Ereignissen ab
 - Bei jedem Testdurchlauf können die Symptome unterschiedlich sein oder verschwinden
- Vermeidung ist u.a durch das Konzept der **Semaphore** ($\implies$ Folie 60) möglich

Therac-25: Race Condition mit tragischem Ausgang (1/2)

- Therac-25 ist ein Elektronen-Linearbeschleuniger zur Strahlentherapie von Krebstumoren
- Verursachte Mitte der 80er Jahre in den USA tödliche Unfälle durch mangelhafte Programmierung und Qualitätssicherung
 - Einige Patienten erhielten eine bis zu hundertfach erhöhte Strahlendosis



Bildquelle: Google Bildersuche.
Häufig gezeigtes Bild in diesem Kontext.
(Autor und Lizenz: unbekannt)

An Investigation of the Therac-25 Accidents. Nancy Leveson, Clark S. Turner. IEEE Computer, Vol. 26, No. 7, July 1993, S.18-41
http://courses.cs.vt.edu/~cs3604/lib/Therac_25/Therac_1.html

Therac-25: Race Condition mit tragischem Ausgang (2/2)

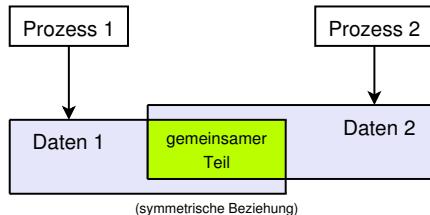
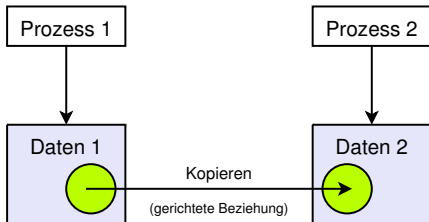
- Eine Race Condition („Texas-Bug“) führte zu fehlerhaften Einstellungen des Geräts und damit zu erhöhter Strahlendosis
 - Der Kontroll-Prozess synchronisierte nicht korrekt mit dem Prozess der Eingabeaufforderung
 - Der Fehler trat nur während einer schnellen Eingabekorrektur (Zeitfenster: 8 Sekunden) durch den Benutzer auf
 - Bei Tests trat der Fehler nicht auf, weil es Erfahrung (Routine) erforderte, um das Gerät so schnell zu bedienen

The Worst Computer Bugs in History: Race conditions in Therac-25:
<https://www.bugsnag.com/blog/bug-day-race-condition-therac-25>

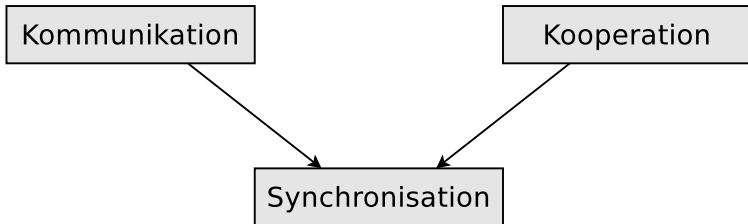
„Once the data entry phase was marked complete, the magnet setting phase began. However, if a specific sequence of edits was applied in the Data Entry phase during the 8 second magnet setting phase, the setting was not applied to the machine hardware, due to the value of the completion variable. The UI would then display the wrong mode to the user, who would confirm the potentially lethal treatment.“

Weitere interessante Quellen

https://www.dssz.informatik.tu-cottbus.de/information/slides_studis/ss2009/mehner_RisikoComputer_zs09.pdf
 Killer Bug. Therac-25: Quick-and-Dirty: https://www.viva64.com/en/b/0438/
 Killed by a machine: The Therac-25: https://hackaday.com/2015/10/26/killed-by-a-machine-the-therac-25/



- Kommunikation und Kooperation basieren auf Synchronisation
 - Synchronisation ist die elementarste Form der Interaktion
 - Grund: Kommunikation und Kooperation benötigen eine zeitliche Abstimmung zwischen den Interaktionspartnern, um korrekte Ergebnisse zu erhalten
 - Darum behandeln wir zuerst die **Synchronisation**



Sperren und Freigeben von Prozessen unter Linux (2/2)

- Alternative 2: Eine lokale Datei dient als Sperrmechanismus für wechselseitigen Ausschluss
 - Jeder Prozess prüft vor dem Eintritt in seinen kritischen Abschnitt, ob er die Datei exklusiv öffnen kann
 - z.B. mit dem Systemaufruf `open` oder der Bibliotheksfunktion `fopen`
 - Ist das nicht der Fall, muss er für eine bestimmte Zeit pausieren (z.B. mit dem Systemaufruf `sleep`) und es danach erneut versuchen (**aktives Warten**)
 - Alternativ kann er sich mit `sleep` oder `pause` selbst pausieren und hoffen, dass der Prozess, der bereits die Datei geöffnet hat ihn nach Abschluss seines kritischen Abschnitts mit einem Signal `deblociert` (**passives Warten**)

Zusammenfassung: Unterschied zwischen Signalisieren und Blockieren

- **Signalisieren** legt die Ausführungsreihenfolge fest
Beispiel: Abschnitt X von Prozess P_A vor Abschnitt Y von P_B ausführen
- **Sperren / Blockieren** sichert kritische Abschnitte
Die Reihenfolge, in der die Prozesse ihre kritische Abschnitte abarbeiten, ist nicht festgelegt! Es wird nur sichergestellt, dass es keine Überlappung in der Ausführung der kritischen Abschnitte gibt

Deadlock-Erkennung mit Matrizen

- Ein Nachteil der Deadlock-Erkennung mit Betriebsmittel-Graphen ist, dass man damit nur einzelne Ressourcen darstellen kann
 - Gibt es mehrere Kopien (Instanzen) einer Ressource, sind Graphen zur Darstellung bzw. Erkennung von Deadlocks ungeeignet
 - Existieren von einer Ressource mehrere Instanzen, kann ein matrizenbasiertes Verfahren verwendet werden, das 2 Vektoren und 2 Matrizen benötigt
- Wir definieren 2 Vektoren
 - **Ressourcenvektor** (*Existing Resource Vektor*)
 - Zeigt an, wie viele Ressourcen von jeder Klasse existieren
 - **Ressourcenrestvektor** (*Available Resource Vektor*)
 - Zeigt an, wie viele Ressourcen von jeder Klasse frei sind
- Zusätzlich sind 2 Matrizen nötig
 - **Belegungsmatrix** (*Current Allocation Matrix*)
 - Zeigt an, welche Ressourcen die Prozesse aktuell belegen
 - **Anforderungsmatrix** (*Request Matrix*)
 - Zeigt an, welche Ressourcen die Prozesse gerne hätten

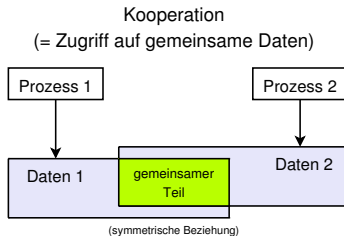
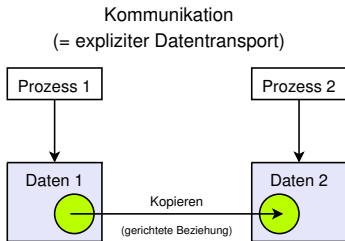
Fazit zu Deadlocks

- Manchmal wird die Möglichkeit von Deadlocks akzeptiert
 - Entscheidend ist, wie wichtig ein System ist
 - Ein Deadlock, der statistisch alle 5 Jahre auftritt, ist kein Problem in einem System das wegen Hardwareausfällen oder sonstigen Softwareproblemen jede Woche ein mal abstürzt
- Deadlock-Erkennung ist aufwendig und verursacht Overhead
- In allen Betriebssystemen sind Deadlocks möglich:
 - Prozesstabelle voll
 - Es können keine neuen Prozesse erzeugt werden
 - Maximale Anzahl von Inodes vergeben
 - Es können keine neuen Dateien und Verzeichnisse angelegt werden
- Die Wahrscheinlichkeit, dass so etwas passiert, ist gering, aber $\neq 0$
 - Solche potentiellen Deadlocks werden akzeptiert, weil ein gelegentlicher Deadlock nicht so lästig ist, wie die ansonsten nötigen Einschränkungen (z.B. nur 1 laufender Prozess, nur 1 offene Datei, mehr Overhead)

Kommunikation von Prozessen

- Kommunikation

- Gemeinsamer Speicher (Shared Memory)
- Nachrichtenwarteschlangen (Message Queues)
- Pipes
- Sockets



Mit gemeinsamem Speicher arbeiten (System V vs. POSIX)

Linux/UNIX-Betriebssysteme stellen 4 Systemaufrufe (System V) für die Arbeit mit gemeinsamem Speicher bereit

- `shmget()`: Gemeinsames Speichersegment erzeugen oder auf ein bestehendes zugreifen
- `shmat()`: Gemeinsames Speichersegment an einen Prozess anhängen
- `shmdt()`: Gemeinsames Speichersegment von einem Prozess lösen/freigeben
- `shmctl()`: Status (u.a. Zugriffsrechte) eines gemeinsamen Speichersegments abfragen, ändern oder es löschen
- Informationen über bestehende gemeinsame Speichersegmente (System V) liefert das Kommando `ipcs`

Ein Beispiel zu gemeinsamen Speicherbereichen (**System V**) unter Linux finden auf der Webseite der Vorlesung

- Einige Entwickler bevorzugen die System V API und andere die POSIX API. . . ㄟ(ㄣ) ㄟ

C-Funktionsaufrufe für gemeinsame POSIX-Speichersegmente (teilweise in der Header-Datei `mman.h` definiert)

- `shm_open()`: Ein Segment erzeugen oder auf ein bestehendes zugreifen
- `ftruncate()`: Die Größe eines Speichersegments definieren
- `mmap()`: Ein Segment an einen Prozess anhängen
- `munmap()`: Ein Segment von einem Prozess lösen/freigeben
- `close()`: Den Deskriptor eines Speichersegments schließen
- `shm_unlink()`: Ein Segment löschen
- Unter Linux liegen POSIX-Speichersegmente im Verzeichnis `/dev/shm`

Ein Beispiel zu gemeinsamen POSIX-Speichersegmenten unter Linux finden sie auf der Webseite der Vorlesung

Gemeinsames Speichersegment (System V) erzeugen (in C)

```
1 #include <sys/ipc.h>
2 #include <sys/shm.h>
3 #include <stdio.h>
4 #define MAXMEMSIZE 20
5
6 int main(int argc, char **argv) {
7     int shared_memory_id = 12345;
8     int returncode_shmget;
9
10    // Gemeinsames Speichersegment erzeugen
11    // IPC_CREAT = Speichersegment erzeugen, wenn es noch nicht existiert
12    // 0600 = Zugriffsrechte auf das neue gemeinsame Speichersegment
13    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
14
15    if (returncode_shmget < 0) {
16        printf("Das gemeinsame Speichersegment konnte nicht erstellt werden.\n");
17        perror("shmget");
18    } else {
19        printf("Das gemeinsame Speichersegment wurde erstellt.\n");
20    }
21 }
```

```
$ ipcs -m
```

```
----- Shared Memory Segments -----
```

key	shmid	owner	perms	bytes	nattch	status
0x00003039	56393780	bnc	600	20	0	

```
$ printf "%d\n" 0x00003039      # Umrechnen von Hexadezimal in Dezimal
12345
```

Gemeins. Speichersegment (System V) anhängen (in C)

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    char *sharedmempointer;
11
12    // Gemeinsames Speichersegment erzeugen
13    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
14    ...
15
16    // Gemeinsames Speichersegment anhängen
17    sharedmempointer = shmat(returncode_shmget, 0, 0);
18    if (sharedmempointer==(char *)-1) {
19        printf("Das gemeinsame Speichersegment konnte nicht angehängt werden.\n");
20        perror("shmat");
21    } else {
22        printf("Das Segment wurde angehängt an Adresse %p\n", sharedmempointer);
23    }
24 }
25 }
```

```
$ ipcs -m
```

```
----- Shared Memory Segments -----
```

key	shmid	owner	perms	bytes	nattch	status
0x00003039	56393780	bnc	600	20	1	

```

1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget, returncode_shmctl, returncode_sprintf;
10    char *sharedmempointer;
11
12    // Gemeinsames Speichersegment erzeugen
13    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
14    ...
15    // Gemeinsames Speichersegment anhängen
16    sharedmempointer = shmat(returncode_shmget, 0, 0);
17    ...
18
19    // Eine Zeichenkette in das gemeinsame Speichersegment schreiben
20    returncode_sprintf = sprintf(sharedmempointer, "Hallo Welt.");
21    if (returncode_sprintf < 0) {
22        printf("Der Schreibzugriff ist fehlgeschlagen.\n");
23    } else {
24        printf("%i Zeichen in das Segment geschrieben.\n", returncode_sprintf);
25    }
26
27    // Die Zeichenkette im gemeinsamen Speichersegment ausgeben
28    if (printf ("%s\n", sharedmempointer) < 0) {
29        printf("Der Lesezugriff ist fehlgeschlagen.\n");
30    }
31    ...

```

Gemeinsames Speichersegment (System V) lösen (in C)

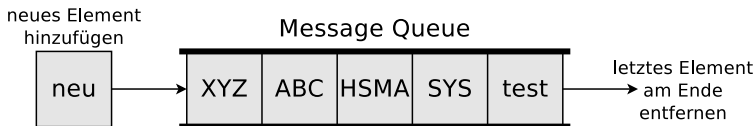
```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    int returncode_shmdt;
11    char *sharedmempointer;
12
13    // Gemeinsames Speichersegment erzeugen
14    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
15    ...
16
17    // Gemeinsames Speichersegment anhängen
18    sharedmempointer = shmat(returncode_shmget, 0, 0);
19    ...
20
21    // Gemeinsames Speichersegment lösen
22    returncode_shmdt = shmdt(sharedmempointer);
23    if (returncode_shmdt < 0) {
24        printf("Das gemeinsame Speichersegment konnte nicht gelöst werden.\n");
25        perror("shmdt");
26    } else {
27        printf("Das Segment wurde vom Prozess gelöst.\n");
28    }
29 }
30 }
```

Gemeinsames Speichersegment (System V) löschen (in C)

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/shm.h>
4 #include <stdio.h>
5 #define MAXMEMSIZE 20
6
7 int main(int argc, char **argv) {
8     int shared_memory_id = 12345;
9     int returncode_shmget;
10    int returncode_shmctl;
11    char *sharedmempointer;
12
13    // Gemeinsames Speichersegment erzeugen
14    returncode_shmget = shmget(shared_memory_id, MAXMEMSIZE, IPC_CREAT | 0600);
15    ...
16
17    // Gemeinsames Speichersegment löschen
18    returncode_shmctl = shmctl(returncode_shmget, IPC_RMID, 0);
19    if (returncode_shmctl == -1) {
20        printf("Das gemeinsame Speichersegment konnte nicht gelöscht werden.\n");
21        perror("semctl");
22    } else {
23        printf("Das Segment wurde gelöscht.\n");
24    }
25 }
26 }
```

Nachrichtenwarteschlangen - Message Queues

- Sind verketteten Listen mit Nachrichten
- Arbeiten nach dem Prinzip FIFO
 - Die Entnahmereihenfolge kann aber gesteuert werden
 - System-V-Warteschlangen können Nachrichten nach Typ auswählen
 - POSIX-Warteschlangen berücksichtigen Nachrichtenprioritäten
- Prozesse können Daten darin ablegen und daraus abholen
- Vorteil: Auch nach Beendigung des Erzeuger-Prozesses verbleiben die Daten in der Nachrichtenwarteschlange



Linux/UNIX-Betriebssysteme stellen 4 Systemaufrufe (System V) für die Arbeit mit Nachrichtenwarteschlangen bereit

- `msgget()`: Nachrichtenwarteschlange erzeugen oder auf eine bestehende zugreifen
- `msgsnd()`: Nachricht in Nachrichtenwarteschlange schreiben (schicken)
- `msgrcv()`: Nachricht aus Nachrichtenwarteschlange lesen (empfangen)
- `msgctl()`: Status (u.a. Zugriffsrechte) einer Nachrichtenwarteschlange abfragen, ändern oder sie löschen
- Informationen über bestehende Nachrichtenwarteschlangen (System V) liefert das Kommando `ipcs`

Nachrichtewarteschlangen (System V) erzeugen (in C)

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6
7 int main(int argc, char **argv) {
8     int returncode_msgget;
9
10    // Nachrichtewarteschlange erzeugen oder auf eine bestehende zugreifen
11    // IPC_CREAT => neue Nachrichtewarteschlange erzeugen, wenn sie noch nicht existiert
12    // 0600 = Zugriffsrechte auf die neue Nachrichtewarteschlange
13    returncode_msgget = msgget(12345, IPC_CREAT | 0600);
14    if(returncode_msgget < 0) {
15        printf("Die Nachrichtewarteschlange konnte nicht erstellt werden.\n");
16        exit(1);
17    } else {
18        printf("Die Nachrichtewarteschlange 12345 mit der ID %i ist nun verfügbar.\n",
19               returncode_msgget);
20    }
```

```
$ ipcs -q
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00003039  98304      bnc        600         0             0

$ printf "%d\n" 0x00003039      # Umrechnen von Hexadezimal in Dezimal
12345
```


Ergebnis des Schreibens in die Nachrichtenwarteschlange

- Vorher...

```
$ ipcs -q
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00003039  98304      bnc        600         0              0
```

- Nachher...

```
$ ipcs -q
----- Message Queues -----
key          msqid      owner      perms      used-bytes   messages
0x00003039  98304      bnc        600         80             1
```

Aus Nachrichtenwarteschlangen (System V) lesen (in C)

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6 #include <string.h>           // Diese Header-Datei ist nötig für strcpy()
7 typedef struct msgbuf {      // Template eines Puffers fuer msgsnd und msgrcv
8     long mtype;              // Nachrichtentyp
9     char mtext[80];          // Sendepuffer
10 } msg;
11
12 int main(int argc, char **argv) {
13     int returncode_msgget, returncode_msgrcv;
14     msg receivebuffer;        // Einen Empfangspuffer anlegen
15
16     // Nachrichtenwarteschlange erzeugen oder auf eine bestehende zugreifen
17     returncode_msgget = msgget(12345, IPC_CREAT | 0600);
18
19     msg.mtype = 1;            // Die erste Nachricht vom Typ 1 empfangen
20     // MSG_NOERROR => Nachrichten abschneiden, wenn sie zu lang sind
21     // IPC_NOWAIT  => Prozess nicht blockieren, wenn keine Nachricht vom Typ vorliegt
22     returncode_msgrcv = msgrcv(returncode_msgget, &msg, sizeof(msg.mtext),
23                                msg.mtype, MSG_NOERROR | IPC_NOWAIT);
24     if (returncode_msgrcv < 0) {
25         printf("Aus der Nachrichtenwarteschlange konnte nicht gelesen werden.\n");
26         perror("msgrcv");
27     } else {
28         printf("Diese Nachricht wurde aus der Warteschlange gelesen: %s\n", msg.mtext);
29         printf("Die empfangene Nachricht ist %i Zeichen lang.\n", returncode_msgrcv);
30     }
31 }
```

Nachrichtenwarteschlangen (System V) löschen (in C)

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <stdio.h>
5 #include <sys/msg.h>
6
7 int main(int argc, char **argv) {
8     int returncode_msgget;
9     int returncode_msgctl;
10
11     // Nachrichtenwarteschlange erzeugen oder auf eine bestehende zugreifen
12     returncode_msgget = msgget(12345, IPC_CREAT | 0600);
13     ...
14
15     // Nachrichtenwarteschlange löschen
16     returncode_msgctl = msgctl(returncode_msgget, IPC_RMID, 0);
17     if (returncode_msgctl < 0) {
18         printf("Die Nachrichtenwarteschlange mit der ID %i konnte nicht
19             gelöscht werden.\n", returncode_msgget);
20         perror("msgctl");
21         exit(1);
22     } else {
23         printf("Die Nachrichtenwarteschlange mit der ID %i wurde
24             gelöscht.\n", returncode_msgget);
25     }
26     exit(0);
27 }
```

Ein Beispiel zur Arbeit mit System V-Nachrichtenwarteschlangen unter Linux finden sie auf der Webseite der Vorlesung

Nachrichtenwarteschl. unter Linux (System V vs. POSIX)

- Die bislang beschriebenen Funktionen zur Arbeit mit Nachrichtenwarteschlangen sind Teil der **System V**-Schnittstelle
- Einige Entwickler bevorzugen die System V API und andere die POSIX API... ㄟ(͜ʖ)ㄟ

In der Header-Datei `mqueue.h` definierte C-Funktionsaufrufe der POSIX-Nachrichtenwarteschlangen

- `mq_open()`: Nachrichtenwarteschlange erzeugen oder auf eine bestehende zugreifen
- `mq_send()`: Nachricht in eine Nachrichtenwarteschlange schreiben (schicken). Blockierende Anweisung
- `mq_timedsend()`: Nachricht in eine Nachrichtenwarteschlange schreiben (schicken). Blockierende Anweisung mit Timeout
- `mq_receive()`: Nachricht aus einer Nachrichtenwarteschlange lesen (empfangen). Blockierende Anweisung
- `mq_timedreceive()`: Nachricht aus einer Nachrichtenwarteschlange lesen (empfangen). Blockierende Anweisung mit Timeout
- `mq_getattr()`: Eigenschaften einer Nachrichtenwarteschlange abfragen. Diese sind: Anzahl der Nachrichten in der Warteschlange, maximale Nachrichtengröße, maximale Anzahl an Nachrichten, etc.
- `mq_setattr()`: Eigenschaften einer Nachrichtenwarteschlange ändern
- `mq_notify()`: Der Prozess soll benachrichtigt werden, sobald eine Nachricht vorliegt
- `mq_close()`: Nachrichtenwarteschlange schließen
- `mq_unlink()`: Nachrichtenwarteschlange löschen
- Unter Linux liegen POSIX-Nachrichtenwarteschlangen im Verzeichnis `/dev/mqueue`

Ein Beispiel zur Arbeit mit benannten POSIX-Nachrichtenwarteschlangen unter Linux finden sie auf der Webseite der Vorlesung

Anonyme Pipes (1/2)

- Pipes können **anonyme** oder **benannte Pipes** (siehe Folie 44) sein
- Eine **anonyme Pipe**. . .
 - ist ein gepufferter unidirektionaler Kanal zwischen 2 Prozessen
 - Soll Kommunikation in beide Richtungen gleichzeitig möglich sein, sind 2 Pipes nötig – eine für jede mögliche Kommunikationsrichtung
 - arbeitet nach dem FIFO-Prinzip
 - hat eine begrenzte Kapazität
 - Pipe = voll $\implies$ der in die Pipe schreibende Prozess wird blockiert
 - Pipe = leer $\implies$ der aus der Pipe lesende Prozess wird blockiert
 - wird mit dem Systemaufruf `pipe()` angelegt
 - Dabei erzeugt der Betriebssystemkern einen Inode ($\implies$ Foliensatz 6) und 2 Zugriffskennungen (*Handles*)
 - Prozesse greifen auf die Zugriffskennungen mit `read()` und `write()`-Systemaufrufen (oder Bibliotheksfunktionen) zu, um Daten aus der Pipe zu lesen bzw. um Daten in die Pipe zu schreiben



Anonyme Pipes (2/2)

- Bei der Erzeugung von Kindprozessen mit `fork()` erben die Kindprozesse auch den Zugriff auf die Zugriffskennungen
- **Anonyme Pipes** ermöglichen Prozesskommunikation nur zwischen eng verwandten Prozessen
 - Nur Prozesse, die via `fork()` eng verwandt sind, können über anonyme Pipes kommunizieren
 - Mit der Beendigung des letzten Prozesses, der Zugriff auf eine anonyme Pipe hat, wird diese vom Betriebssystem beendet

Übersicht der Pipes unter Linux/UNIX: `lsuf` | `grep pipe`

Standardgröße und maximale Größe von anonymen Pipes unter Linux

- Standardgröße $\implies$ `int default_size = fcntl(pipefd[1], F_GETPIPE_SZ);`
Typischerweise in Linux: 65536 Bytes
- Standardgröße ändern $\implies$ `fcntl(pipefd[1], F_SETPIPE_SZ, <size_in_bytes>);`
- Die maximale erlaubte Größe (in Bytes) ist in dieser Datei definiert: `/proc/sys/fs/pipe-max-size`
Typischerweise in Linux: 1048576 = 1 MB

Quelle: <https://manpages.ubuntu.com/manpages/jammy/en/man7/pipe.7.html>

Ein Beispiel zu anonymen Pipes (in C) – Teil 1/2

Sie können die anonyme Pipe unter Linux/UNIX mit `ls -l | grep <PID>` und im Verzeichnis `/proc/<PID>/fd` überwachen

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 void main() {
6     int pid_des_Kindes;
7     // Zugriffskennungen zum Lesen (testpipe[0]) und Schreiben (testpipe[1]) anlegen
8     int testpipe[2];
9
10    // Die Pipe testpipe anlegen
11    if (pipe(testpipe) < 0) {
12        printf("Das Anlegen der anonymen Pipe ist fehlgeschlagen.\n");
13        // Programmabbruch
14        exit(1);
15    } else {
16        printf("Die anonyme Pipe testpipe wurde angelegt.\n");
17    }
18
19    // Einen Kindprozess erzeugen
20    pid_des_Kindes = fork();
21
22    if (pid_des_Kindes < 0) {
23        perror("Es kam bei fork zu einem Fehler!\n");
24        // Programmabbruch
25        exit(1);
26    }
```

Ein Beispiel zu anonymen Pipes (in C) – Teil 2/2

```
27 // Elternprozess
28 if (pid_des_Kindes > 0) {
29     printf("Elternprozess: PID: %i\n", getpid());
30     // Lesekanal der Pipe testpipe blockieren
31     close(testpipe[0]);
32     char nachricht[] = "Testnachricht";
33     // Daten in den Schreibkanal der Pipe schreiben
34     write(testpipe[1], &nachricht, sizeof(nachricht));
35 }
36
37 // Kindprozess
38 if (pid_des_Kindes == 0) {
39     printf("Kindprozess: PID: %i\n", getpid());
40     // Schreibkanal der Pipe testpipe blockieren
41     close(testpipe[1]);
42     // Einen Empfangspuffer (80 Zeichen Kapazität) anlegen
43     char puffer[80];
44     // Daten aus dem Lesekanal der Pipe auslesen
45     read(testpipe[0], puffer, sizeof(puffer));
46     printf("Empfangen: %s\n", puffer);
47 }
48 }
```

```
$ gcc anonyme_pipe_beispiel.c -o anonyme_pipe_beispiel
$ ./anonyme_pipe_beispiel
Die anonyme Pipe testpipe wurde angelegt.
Elternprozess: PID: 6363
Kindprozess: PID: 6364
Empfangen: Testnachricht
```

Benannte Pipes

- Via **benannte Pipes** (Named Pipes), können auch nicht eng miteinander verwandte Prozesse kommunizieren
 - Auf diese Pipes kann mit Hilfe ihres Namens zugegriffen werden
 - Sie werden in C erzeugt via: `mkfifo("<pfadname>", <zugriffsrechte>)`
 - Jeder Prozess, der den Namen kennt, kann über diesen die Verbindung zur Pipe herstellen und darüber mit anderen Prozessen kommunizieren
- **Wechselseitigen Ausschluss** garantiert das Betriebssystem
 - Zu jedem Zeitpunkt kann nur 1 Prozess auf eine Pipe zugreifen
- Benannte Pipes werden vom Betriebssystem nicht automatisch gelöscht (im Gegensatz zu anonymen Pipes)

Größe benannter Pipes in Linux

- Die maximale Anzahl von Bytes, die atomar (ohne Verschachtelung mit anderen Schreibvorgängen) geschrieben werden können, definiert die Systemvariable `PIPE_BUF` und entspricht normalerweise der Seitengröße $\Rightarrow$ `getconf PIPE_BUF /path`
Typischerweise in Linux: 4096 Bytes in einem System mit 4096 Bytes Seitengröße
- Die Größe (in Bytes) wird vom Kernel festgelegt
Typischerweise in Linux: 65536 Bytes in einem System mit 4096 Bytes Seitengröße
- Die Größe kann nicht dynamisch geändert werden. `F_SETPIPE_SZ` kann nicht mit benannten Pipes verwendet werden. Die Größe der benannten Pipes wird vom Kernel festgelegt und nicht von der Datei `/proc/sys/fs/pipe-max-size`

Ein Beispiel zu benannten Pipes (in C) – Teil 1/4

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <fcntl.h>
5 #include <sys/stat.h>
6
7 void main() {
8     int pid_des_Kindes;
9
10    // Die benannte Pipe anlegen
11    if (mkfifo("testfifo",0666) < 0) {
12        printf("Das Anlegen der benannten Pipe ist fehlgeschlagen.\n");
13        exit(1);
14    } else {
15        printf("Die benannte Pipe testfifo wurde angelegt.\n");
16    }
17
18    // Einen Kindprozess erzeugen
19    pid_des_Kindes = fork();
20
21    if (pid_des_Kindes < 0) {
22        perror("Es kam bei fork zu einem Fehler!\n");
23        exit(1);
24    }
```

Der Funktionsaufruf erzeugt im aktuellen Verzeichnis einen Dateisystemeintrag mit dem Namen testfifo. Der erste Buchstabe in der Ausgabe des Kommandos ls zeigt, dass testfifo eine benannte Pipe ist. Die Zugriffsrechte sind rw-r--r-- weil umask ist 022.

```
$ ls -la testfifo
prw-r--r--  1 bnc bnc    0  1. Feb 10:15 testfifo
```

Ein Beispiel zu benannten Pipes (in C) – Teil 2/4

```
25 // Elternprozess
26 if (pid_des_Kindes > 0) {
27     printf("Elternprozess: PID: %i\n", getpid());
28
29     // Zugriffskennung für die benannte Pipe anlegen
30     int fd;
31
32     // Die zu übertragene Nachricht definieren
33     char nachricht[] = "Testnachricht";
34
35     // Die benannte Pipe für Schreibzugriffe öffnen
36     fd = open("testfifo", O_WRONLY);
37
38     // Daten in die benannte Pipe schreiben
39     write(fd, &nachricht, sizeof(nachricht));
40
41     // Die benannte Pipe schließen
42     close(fd);
43 }
```

Ein Beispiel zu benannten Pipes (in C) – Teil 3/4

```
44 // Kindprozess
45 if (pid_des_Kindes == 0) {
46     printf("Kindprozess: PID: %i\n", getpid());
47
48     // Zugriffskennung für die benannte Pipe anlegen
49     int fd;
50     // Einen Empfangspuffer anlegen
51     char puffer[80];
52
53     // Die benannte Pipe für Lesezugriffe öffnen
54     fd = open("testfifo", O_RDONLY);
55
56     // Daten aus der Pipe auslesen
57     read(fd, puffer, sizeof(puffer));
58     printf("Empfangen: %s\n", puffer);
59
60     // Die beannte Pipe schließen
61     close(fd);
62
63     // Die benannte Pipe löschen
64     if (unlink("testfifo") < 0) {
65         printf("Das Löschen der benannten Pipe ist fehlgeschlagen.\n");
66         exit(1);
67     } else {
68         printf("Die benannte Pipe wurde gelöscht.\n");
69     }
70 }
71 }
```

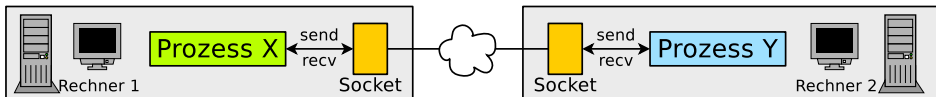
Ein Beispiel zu benannten Pipes (in C) – Teil 4/4

```
$ gcc benannte_pipe_beispiel.c -o benannte_pipe_beispiel
$ ./benannte_pipe_beispiel
Die benannte Pipe testfifo wurde angelegt.
Elternprozess: PID: 403660
Kindprozess: PID: 403661
Empfangen: Testnachricht
Die benannte Pipe wurde gelöscht.
```

Sie können die benannte Pipe unter Linux/UNIX mit `ls -l | grep <PID>` und im Verzeichnis `/proc/<PID>/fd` überwachen

Sockets

- Vollduplexfähige Alternative zu Pipes und gemeinsamem Speicher
 - Ermöglichen Interprozesskommunikation in verteilten Systemen
- Ein Benutzerprozess kann einen Socket vom Betriebssystem anfordern, und über diesen anschließend Daten verschicken und empfangen
 - Das Betriebssystem verwaltet alle benutzten Sockets und die zugehörigen Verbindungsinformationen



- Zur Kommunikation über Sockets werden Ports verwendet
 - Die Vergabe der Portnummern erfolgt beim Verbindungsaufbau
 - Portnummern werden vom Betriebssystem zufällig vergeben
 - Ausnahmen sind Ports bekannter Anwendungen, wie z.B. HTTP (80), SMTP (25), Telnet (23), SSH (22), FTP (21),...
- Einsatz von Sockets ist blockierend (synchron) und nicht-blockierend (asynchron) möglich

Verschiedene Arten von Sockets

• Verbindungslose Sockets (bzw. Datagram Sockets)

- Verwenden das Transportprotokoll UDP
- Vorteil: Höhere Geschwindigkeit als bei TCP
 - Grund: Geringer Mehraufwand (Overhead) für das Protokoll
- Nachteil: Segmente können einander überholen oder verloren gehen

• Verbindungsorientierte Sockets (bzw. Stream Sockets)

- Verwenden das Transportprotokoll TCP
- Vorteil: Höhere Verlässlichkeit
 - Segmente können nicht verloren gehen
 - Segmente kommen immer in der korrekten Reihenfolge an
- Nachteil: Geringere Geschwindigkeit als bei UDP
 - Grund: Höherer Mehraufwand (Overhead) für das Protokoll

Sockets nutzen

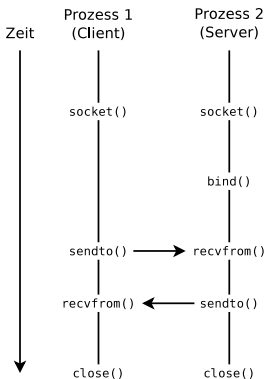
- Praktisch alle gängigen Betriebssystemen unterstützen Sockets
 - Vorteil: Bessere Portabilität der Anwendungen
- Funktionen für Kommunikation via Sockets:
 - Erstellen eines Sockets:
`socket()`
 - Anbinden eines Sockets an eine Portnummer und empfangsbereit machen:
`bind()`, `listen()`, `accept()` und `connect()`
 - Senden/Empfangen von Nachrichten über den Socket:
`send()`, `sendto()`, `recv()` und `recvfrom()`
 - Schließen eines Sockets (Dateideskriptor freigeben):
`close()`

Zusätzlich zu den genannten Funktionen gibt es noch `shutdown()`, um eine oder beide Kommunikationsrichtungen zu beenden. `SHUT_RD` = keine weiteren Daten empfangen, `SHUT_WR` = keine weiteren Daten senden, `SHUT_RDWR` = beide Richtungen deaktivieren. Im Gegensatz zur Funktion `close()` wird der Dateideskriptor aber nicht freigegeben.

Übersicht der Sockets unter Linux/UNIX: `netstat -n` oder `lsof | grep socket`

Beispiele zur Interprozesskommunikation via Sockets (TCP and UDP) unter Linux finden Sie auf der Webseite der Vorlesung

Verbindungslose Kommunikation mit Sockets – UDP



• Client

- Socket erstellen (socket)
- Daten senden (sendto) und empfangen (recvfrom)
- Socket schließen (close)

• Server

- Socket erstellen (socket)
- Socket an einen Port binden (bind)
- Daten senden (sendto) und empfangen (recvfrom)
- Socket schließen (close)

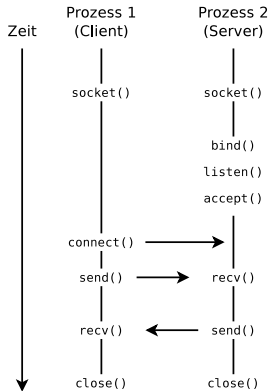
Verbindungsorientierte Kommunikation mit Sockets – TCP

• Client

- Socket erstellen (`socket`)
- Client mit Server-Socket verbinden (`connect`)
- Daten senden (`send`) und empfangen (`recv`)
- Socket schließen (`close`)

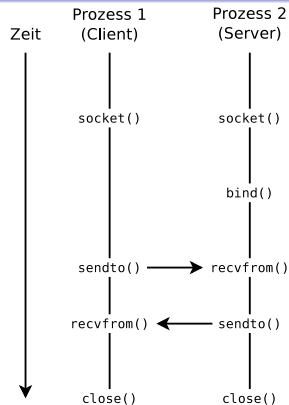
• Server

- Socket erstellen (`socket`)
- Socket an einen Port binden (`bind`)
- Socket empfangsbereit machen (`listen`)
 - Warteschlange für Verbindungsanfragen einrichten. Definiert wie viele Verbindungsanfragen gepuffert werden können
- Verbindungsanforderung akzeptieren (`accept`)
 - Erste Verbindungsanforderung aus der Warteschlange holen
- Daten senden (`send`) und empfangen (`recv`)
- Socket schließen (`close`)



Sockets via UDP – Beispiel (Server)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6 #include <unistd.h>
7 #include <arpa/inet.h>
8
9 int main(int argc, char *argv[]) {
10     int sd, adresse_laenge;
11     char puffer[1024] = { 0 };
12     struct sockaddr_in adresse, client_adresse;
13     memset(&adresse, 0, sizeof(adresse));
14     memset(&client_adresse, 0, sizeof(client_adresse));
15     adresse.sin_family = AF_INET;
16     adresse.sin_addr.s_addr = INADDR_ANY;
17     adresse.sin_port = htons(atoi(argv[1]));
18
19     sd = socket(AF_INET, SOCK_DGRAM, 0);
20     bind(sd, (struct sockaddr *) &adresse, sizeof(adresse));
21     adresse_laenge = sizeof(client_adresse);
22     recvfrom(sd, (char *)puffer, sizeof(puffer), 0,
23             (struct sockaddr *) &client_adresse, &adresse_laenge);
24     printf("Empfangene Nachricht: %s\n", puffer);
25     char antwort[]="Server: Nachricht empfangen.\n";
26     sendto(sd, (const char *)antwort, sizeof(antwort), 0,
27           (struct sockaddr *) &client_adresse, adresse_laenge);
28     close(sd);
29     exit(0);
30 }
```



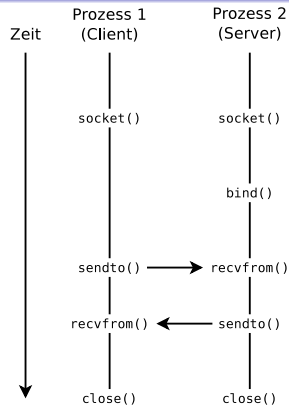
```
$ gcc udp_server.c -o udp_server
$ ./udp_server 50002
```

Sockets via UDP – Beispiel (Client)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6 #include <unistd.h>
7 #include <arpa/inet.h>
8
9 int main(int argc, char *argv[]) {
10     int sd, adresse_laenge;
11     char puffer[1024] = { 0 };
12     struct sockaddr_in adresse;
13     memset(&adresse, 0, sizeof(adresse));
14     adresse.sin_family = AF_INET;
15     adresse.sin_port = htons(atoi(argv[2]));
16     adresse.sin_addr.s_addr = inet_addr(argv[1]);
17
18     sd = socket(AF_INET, SOCK_DGRAM, 0);
19     printf("Bitte Nachricht eingeben: ");
20     fgets(puffer, sizeof(puffer), stdin);
21     adresse_laenge = sizeof(adresse);
22     sendto(sd, (const char *)puffer, strlen(puffer), 0,
23             (struct sockaddr *) &adresse, adresse_laenge);
24     memset(puffer, 0, sizeof(puffer));
25     recvfrom(sd, (char *)puffer, sizeof(puffer), 0,
26              (struct sockaddr *) &adresse, &adresse_laenge);
27     printf("%s\n", puffer);
28     close(sd);
29     exit(0);
30 }

```



```
$ gcc udp_client.c -o udp_client
$ ./udp_client 127.0.0.1 50002
Bitte Nachricht eingeben: Test
Server: Nachricht empfangen.
```

```
$ ./udp_server 50002
Empfangene Nachricht: Test
```


Vergleich der Kommunikations-Systeme

	Gemeinsamer Speicher	Nachrichtenwarteschlangen	anonyme Pipes	benannte Pipes	Sockets
Speicherbasierte oder nachrichtenbasierte Kommunikation	Speicher	Nachrichten	Nachrichten	Nachrichten	Nachrichten
Bidirektional	ja	ja	nein	nein	ja
Prozesse müssen verwandt sein	nein	nein	ja	nein	nein
Kommunikation über Rechnergrenzen	nein	nein	nein	nein	ja
Bleiben ohne gebundenen Prozess erhalten	ja	ja	nein	ja	nein
Automatische Synchronisierung der Zugriffe	nein	ja	ja	ja	ja

• Vorteile nachrichtenbasierte vs. speicherbasierte Kommunikation:

- Das Betriebssystem nimmt den Benutzerprozessen die Synchronisation der Zugriffe ab $\implies$ komfortabel
- Einsetzbar in verteilten Systemen ohne gemeinsamen Speicher
- Bessere Portabilität der Anwendungen

Speicher kann über Netzwerkverbindungen eingebunden werden

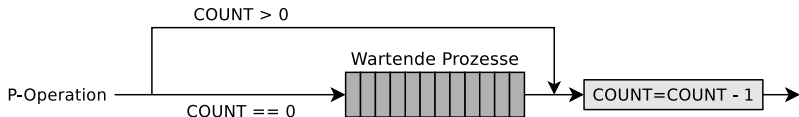
- z.B. durch Nutzung eines Protokolls wie Network File System (NFS) oder Server Message Block (SMB)
- Das ermöglicht speicherbasierte Kommunikation zwischen Prozessen auf verschiedenen, unabhängigen Systemen
- Das Problem der Synchronisation der Zugriffe besteht aber auch hier

Zugriffsoperationen auf Semaphoren (2/3)

Bildquelle: Carsten Vogt

- **P-Operation** (*verringern*): Prüft den Wert der Zählvariable
 - Ist der Wert 0, wird der Prozess blockiert
 - Ist der Wert > 0 , wird er um 1 erniedrigt

```
1 SEM.P() {  
2   // Ist die Zaehlvariable = 0, wird blockiert  
3   if (SEM.COUNT == 0)  
4       < blockiere >  
5  
6   // Ist die Zaehlvariable > 0, wird die  
7   // Zaehlvariable unmittelbar um 1 erniedrigt  
8   SEM.COUNT = SEM.COUNT - 1;  
9 }
```



Michael Vignaro


```

1 typedef int semaphore;           // Semaphore sind von Typ Integer
2 semaphore voll = 0;              // zählt die belegten Plätze im Puffer
3 semaphore leer = 8;              // zählt die freien Plätze im Puffer
4 semaphore mutex = 1;             // steuert Zugriff auf kritische Bereiche
5
6 void erzeuger (void) {
7     int daten;
8
9     while (TRUE) {               // Endlosschleife
10        erzeugeDatenpaket(daten); // erzeuge Datenpaket
11        P(leer);                  // Zähler "leere Plätze" erniedrigen
12        P(mutex);                 // in kritischen Bereich eintreten
13        einfuegenDatenpaket(daten); // Datenpaket in den Puffer schreiben
14        V(mutex);                 // kritischen Bereich verlassen
15        V(voll);                  // Zähler für volle Plätze erhöhen
16    }
17 }
18
19 void verbraucher (void) {
20     int daten;
21
22     while (TRUE) {               // Endlosschleife
23        P(voll);                  // Zähler "volle Plätze" erniedrigen
24        P(mutex);                 // in kritischen Bereich eintreten
25        entferneDatenpaket(daten); // Datenpaket aus dem Puffer holen
26        V(mutex);                 // kritischen Bereich verlassen
27        V(leer);                  // Zähler für leere Plätze erhöhen
28        verbraucheDatenpaket(daten); // Datenpaket nutzen
29    }
30 }

```

Bildquelle: Carsten Vogt

-
- Gruppennummer
- Semaphorentabelle
- 0 1 2 3
- 0 1 2 3 4 5
- S_{00} S_{01} S_{02} S_{03} S_{04} S_{05}
- S_{10} S_{11}
- S_{20} S_{21} S_{22}
- S_{30} S_{31} S_{32} S_{33} S_{34}
- ...
- n leer
- Semaphorennummer innerhalb der Gruppe
- einzelnes Semaphor
- Semaphorengruppe

- `semget()`: Neues Semaphore oder eine Gruppe von Semaphoren erzeugen oder ein bestehendes Semaphore öffnen
- `semctl()`: Wert eines existierenden Semaphors oder einer Semaphorengruppe abfragen, ändern oder ein Semaphore löschen
- `semop()`: P- und V-Operationen auf Semaphoren durchführen
- Informationen über bestehende Semaphore liefert das Kommando `ipcs`

Semaphoren unter Linux (System V vs. POSIX)

IPC-Objekte kontrollieren und löschen

- Informationen über bestehende (**System V**)-Speichersegmente, (**System V**)-Nachrichtenwarteschlangen und (**System V**)-Semaphoren liefert das Kommando `ipcs`
- Die einfachste Möglichkeit, solche gemeinsamen Speichersegmente, Nachrichtenwarteschlangen und Semaphoren auf der Kommandozeile zu löschen, ist das Kommando `ipcrm`

```
ipcrm [-m shmids] [-q msgids] [-s semids]
      [-M shmkeys] [-Q msgkeys] [-S semkeys]
```

- **POSIX**-Speichersegmente und **POSIX**-Semaphoren können im Verzeichnis `/dev/shm` untersucht und von Hand gelöscht werden
- **POSIX**-Nachrichtenwarteschlangen im Verzeichnis `/dev/mqueue` untersucht und von Hand gelöscht werden